CSED233 · 데이터구조 · 포스텍

Ch8. Sorting

1. Sorting 기본 정의

핵심 개념

- **정렬**: 원소 (key, info) 쌍들을 특정 순서로 재배치
- **Linear ordering** 조건:
  - **Trichotomy**: 임의 a, b에 대해 $a < b, a = b, a > b$ 중 **정확히 하나** 성립
  - **Transitivity**: $a < b$ & $b < c \rightarrow a < c$

분류 축

축	종류
메모리	Internal (in-memory) / External (disk)
방법	Comparison / Non-comparison
안정성	Stable / Unstable

Stability(안정성) 정의

- **Stable**: 동일 key 원소들의 **원래 상대 순서 보존**
- 예: $(3, y)(3, x)(4, a)(5, b)$ 정렬
  - Stable → $(3, y)(3, x)(4, a)(5, b)$ (y가 x 앞, 원래 순서)
  - Unstable → $(3, x)(3, y)(4, a)(5, b)$ (바뀜)
- **왜 중요한가**: 다중 키 정렬에서는 보조 키(예: 학번)로 먼저 정렬한 뒤 주 키(예: 점수)로 안정 정렬하면, 주 키가 같은 원소들 사이에서만 보조 키의 정렬 순서가 유지된다. 예를 들어 학번 오름차순 정렬 후 점수 오름차순으로 안정 정렬하면, 최종 결과는 점수 오름차순이며 점수가 같은 학생들은 학번 오름차순으로

배치된다. 따라서 안정 정렬은 이전 정렬 결과 전체가 아니라 두 번째 정렬에서 키가 같은 원소들의 상대 순서를 보존한다.

2. Bubble Sort(버블 정렬)

아이디어

- **k-th pass에서 k-th 최솟값이 k-th 위치로 '떠오름'**
- 인접 원소 비교 → 순서 뒤바뀐 쌍 swap
- **Early termination**: 한 pass에서 swap 0번 → 이미 정렬됨 → 종료
- 방향 명시: 이 노트의 구현은 오른쪽에서 왼쪽으로 순회하는 변형으로, pass마다 최솟값이 왼쪽 끝에 확정된다. 일반적인 오름차순 구현은 왼쪽에서 오른쪽으로 순회하며 왼쪽 키가 크면 교환해 pass마다 최댓값이 뒤쪽에 확정된다. 어느 방향이든 교환이 0회인 pass가 나오면 조기 종료할 수 있다.

복잡도

Case	시간
Best (이미 정렬)	$O(n)$ — early termination
Average / Worst	Θ(n^2) — 최악의 경우 비교 횟수는 $n(n-1)/2$번이고 교환 횟수도 최대 $n(n-1)/2$번

- Bubble sort는 교환 여부에 따른 조기 종료를 적용했을 때 이미 정렬된 입력에서 첫 pass의 $n-1$번 비교만 수행하므로 최선 시간은 Θ(n)이다. 입력 순열이 균등하게 주어진다고 가정하면 평균 시간은 Θ(n^2)이며, 실제 비교 횟수는 입력과 조기 종료 시점에 따라 달라진다. 최악의 경우 비교 횟수는 $n(n-1)/2$번이고 교환 횟수도 최대 $n(n-1)/2$번이므로 최악 시간은 Θ(n^2)이다.
- **용도**: $n < 100$ 정도 소규모만. 실무 거의 안 씀.

3. Selection Sort(선택 정렬)

아이디어

- **k-th pass에서 unsorted 부분의 최솟값 찾아 $A[k]$와 swap**
- **pass당 swap 최대 1번** → swap 비용 중요한 상황에 유리

복잡도

- **비교 횟수는 항상 $O(n^2)$**: best = average = worst
  - 이유: 매 pass마다 나머지 전체를 스캔해 최솟값 탐색
- Selection sort는 각 pass에서 미정렬 구간의 최솟값을 찾아 현재 위치에 배치하므로 pass당 swap 횟수는 최대 1번이고 전체 swap 횟수는 최대 *n*−1번이다. 최솟값이 이미 현재 위치에 있다면 swap을 생략할 수 있으므로 실제 횟수는 더 적을 수 있으며, 이미 정렬된 입력에서는 0번도 가능하다. 따라서 비교 횟수는 입력과 무관하게 *n*(*n*−1)/2번이지만 swap 횟수는 구현과 입력에 따라 달라진다.

Bubble vs Selection

	Bubble	**Selection**
Best case	$O(n)$	$O(n^2)$
Swap count	$O(n^2)$	$O(n)$
Stable?	Yes	No (일반적)

4. Insertion Sort(삽입 정렬)

아이디어

- **k-th pass에서 $A[k]$를 이미 정렬된 $A[1..k-1]$에 올바른 위치에 삽입**
- 카드 정렬 직관: 한 장씩 뽑아 손에 있는 정렬된 카드에 꽂음

복잡도

- **Best $O(n)$**: 이미 정렬된 경우 → 각 원소 비교 1번
- Insertion sort는 현재 원소를 임시 변수에 저장한 뒤 이미 정렬된 앞부분에서 현재 원소보다 큰 원소들을 오른쪽으로 한 칸씩 이동시키고, 비워진 위치에 현재 원소를 삽입한다. 이미 정렬된 입력에서는 각 원소를 한 번씩 확인하므로 최선 시간은 Θ(*n*)이며, 입력 순열이 균등하게 주어질 때 평균 시간과 역순 입력에서의 최악 시간은 모두 Θ(n^2)이다. 이때 기본 shift 구현은 원소 이동을 수행하며 swap을 필수로 사용하지 않는다. 인접 원소를 반복 교환하는 변형에서는 평균·최악 swap 횟수가 Θ(n^2)이 될 수 있다. 같은 키를 가진 기존 원소를 새 원소 뒤로 밀지 않으면 안정성을 유지한다.

실전 가치

- Insertion sort는 시간 복잡도가 평균·최악 Θ(n^2)이지만 구현이 단순하고 반복문의 상수 비용이 작기 때문에 작은 부분배열에서는 Merge sort나 Quick sort보다 빠를 수 있다. 따라서 하이브리드 정렬 구현에서는 재귀적으로 처리할 부분배열의 크기가 일정 임계값 이하가 되면 Insertion sort로 전환하기도 한다. 이 임계값은 이론적으로 15로 고정된 것이 아니라 구현·하드웨어·입력 특성에 따라 달라지며, 실제 성능 측정을 통해 정하는 값이다.
- → **Merge/Quick sort에서 recursion base로 사용** (hybrid 기법)

5. Bucket Sort (Non-Comparison)

아이디어

- 키 값으로 **직접 index** 접근 → 비교 없이 분류
- **Phase 1 (Scatter)**: 각 원소를 key에 해당하는 bucket에 배치
- **Phase 2 (Gather)**: bucket 순서대로 꺼내 연결

복잡도

- Scatter: $O(n)$
- Gather: $O(n + N)$ (bucket 수 N 고려)
- **Total:** $O(n + N)$

적용 조건

- **Key 범위** $[0, N-1]$**이 작을 때** 효율적
- 적용 조건: 정수 키 직접 인덱싱 방식의 Bucket Sort는 n개의 원소를 처리하고 N개의 bucket을 초기화·순회하므로 시간 복잡도가 O(n+N)이다. N=O(n)이면 전체 시간도 O(n)이므로 효율적일 수 있다. 반면 N이 n에 비해 매우 크게 증가하면 빈 bucket의 초기화·순회 비용이 지배적이 되어 비효율적일 수 있다. 따라서 N>n이라는 조건만으로 비교 정렬보다 반드시 느리다고 단정할 수 없으며, 실제 선택은 N의 크기, 메모리 사용량, 다른 정렬 알고리즘의 비용을 함께 고려해야 한다.

정수 키 직접 인덱싱 Bucket Sort의 안정성: 정수 키가 [m, m+N−1] 범위에 있다면 각 원소를 bucket[key−m]에 배치할 수 있다. 이때 원소를 원래 입력 순서대로 각 bucket의 끝에 추가하고, gather 단계에서도 bucket 내부 원소를 삽입된 순서대로 출력하면 같은 키의 상대 순서가 보존되므로 안정 정렬이다. 반대로 bucket 내부에서 입력 순서를 보존하지 않는 구현은 안정성을 보장하지 않는다.

Q1. Float key(예: 0.78, 0.12, …)도 bucket sort가 가능한가?

> 힌트

bucket sort는 key의 값 범위를 구간으로 나누는 방식이다.

> 정답

정답: 가능하다. 모든 키가 [0,1) 범위에 있으면 실수 키의 구간을 k개의 bucket으로 나누어 정렬할 수 있다.

정렬 절차: 각 실수 키 x를 인덱스 $\lfloor kx \rfloor$에 해당하는 bucket에 넣는다. 이후 각 bucket 내부를 Insertion Sort 등의 알고리즘으로 정렬하고, bucket을 인덱스 오름차순으로 연결하면 전체 배열이 정렬된다.

평균 복잡도: n개의 키가 독립적인 균등분포에서 추출되고 bucket 내부를 Insertion Sort로 정렬하는 경우, 전체 기대 시간은 $O(n+k+n^2/k)$이다. 특히 $k=\Theta(n)$이면 기대 시간은 $O(n)$이 된다.

최악 복잡도: 입력이 한 bucket에 집중되고 bucket 내부에서 많은 이동이 필요한 경우에는 Insertion Sort 비용이 $\Theta(n^2)$까지 증가하여 전체 최악 시간이 $O(n^2+k)$가 된다.

핵심 구별: 정수 키 직접 인덱싱 방식의 $O(n+N)$은 키 범위 크기 N에 의해 결정되고, 실수 구간 분할 방식의 기대 $O(n)$은 적절한 bucket 수와 입력 분포 가정에 의존한다. 두 복잡도를 같은 조건에서 성립하는 결과로 취급해서는 안 된다.

Q2. Bucket sort가 느려지는 경우는?

> 힌트

bucket의 개수와 분포 균일성이 성능을 좌우한다.

> 정답

개념은 힌트 참고.

- Bucket Sort의 구현 방식에 따라 성능 저하 조건이 다르다. (1) 정수 키 직접 인덱싱 방식: 키 범위의 크기 N이 n에 비해 매우 크면 빈 bucket의 초기화·순회 비용이 커져 O(n+N) 시간이 지배된다. 각 bucket에 원소를 상수 시간으로 추가하고 그대로 출력하는 구현이라면, 원소가 특정 bucket에 집중되더라도 점근적 시간 상한 O(n+N)은 변하지 않는다. (2) 실수 키 구간 분할 방식: 각 bucket 내부를 Insertion Sort로 정렬한다면 입력 분포가 편중되어 한 bucket에 많은 원소가 몰릴 때 정렬 비용이 증가한다. 최악에는 한 bucket에 n개의 원소가 모여 전체 시간이 $O(n^2+k)$가 된다. 따라서 두 방식의 복잡도를 분석할 때는 각각 키 범위의 크기와 bucket 내부 원소 수의 분포를 구별해야 한다.

6. Merge Sort(병합 정렬)

Divide & Conquer

1. **Divide**: 배열을 반으로 분할
2. **Conquer**: 각 반을 재귀적으로 정렬
3. **Combine**: 두 정렬된 리스트를 **merge**

Merge 연산 자체

- 두 정렬 리스트 A, B를 merge
- Merge 연산은 이미 정렬된 두 리스트의 현재 원소를 두 포인터로 비교하여 키가 작은 원소를 출력에 추가한다. 두 원소의 키가 같은 경우에는 원래 배열에서 앞에 있던 왼쪽 부분배열의 원소를 먼저 출력해야 동일 키의 상대 순서가 보존된다.
- 한쪽 리스트가 소진되면 다른 쪽 리스트의 남은 원소들을 기존 순서대로 출력에 연결한다. 이 동률 처리 규칙을 모든 병합 단계에서 지키면 Merge sort는 안정 정렬이며, 길이가 각각 *n*, *m*인 두 리스트의 병합 시간은 *O*(*n*+*m*)이다.
- **시간:** $O(n+m)$

재귀 트리 & 복잡도 유도

- Level 수: $O(\log n)$
- 각 level에서 **merge 총 작업량 =** $O(n)$ (모든 원소 한 번씩)
- **Total =** $O(n \log n)$

단점 & 최적화

- 배열 기반 Merge sort는 두 정렬된 부분배열을 병합할 때 일반적으로 원소 수에 비례하는 보조 버퍼가 필요하므로 추가 공간 복잡도는 *O*(*n*)이다. 반면 연결 리스트 기반 Merge sort는 노드 사이의 포인터를 재연결하는 방식으로 merge 연산 자체를 *O*(1) 추가 공간에 수행할 수 있으며, 재귀 구현에서는 별도로 *O*(log *n*) 호출 스택 공간이 필요할 수 있다. 외부 병합 정렬은 데이터가 주기억장치에 모두 들어가지 않을 때 디스크의 정렬된 구간들을 제한된 메모리 버퍼로 읽어 병합하는 방식이므로 일반 배열 기반 Merge sort의 보조 공간 설명과 구분해야 한다.
- **Stable** ✓
- **Worst case도** $O(n \log n)$ ✓ → 보장 필요 시 유리

- 최적화: $n \leq 15$일 때는 insertion sort 사용 (상수항 작음)

Non-recursive (Bottom-up)

- Size 1 segment에서 시작 → pairwise merge → size 2 → size 4 → ...
- Downward pass 제거, 직접 위로 병합

7. Quick Sort(퀵 정렬)

Divide & Conquer (Partition 기반)

1. **Pivot(피벗) 선택**
2. **Partition**: pivot 기준 3 그룹
   - Left: key < pivot
   - Pivot
   - Right: key ≥ pivot
3. **재귀**: left, right 각각 정렬
4. **Combine**: Quick sort에서 sorted(L) + pivot + sorted(R)은 최종 결과의 논리적인 순서를 나타내는 표현이다. 제자리 Quick sort는 partition 과정에서 pivot을 최종 위치에 배치한 뒤 그 양쪽 부분배열을 같은 배열 안에서 정렬하므로 별도의 concat 연산이 필요하지 않으며, 이 의미에서 combine 단계의 추가 작업은 $O(1)$이다. 반면 별도의 리스트들을 실제로 복사하여 연결하는 구현에서는 결합에 $O(n)$ 시간과 추가 저장 공간이 필요할 수 있다.

Partition 방법

양방향 In-place Partition — 첫 원소를 pivot으로 사용하는 구현

- 배열의 현재 정렬 구간을 A[*lo*..*hi*]라 하고 pivot 값 p=A[*lo*]를 선택한다. 왼쪽 포인터를 L=*lo*+1, 오른쪽 포인터를 R=*hi*로 초기화한다.
- 왼쪽 포인터 L은 배열 범위 안에서 A[L]<p인 원소들을 건너뛰며 오른쪽으로 이동하고, A[L]≥p인 원소를 만나면 멈춘다.
- 오른쪽 포인터 R의 이동: R이 lo보다 큰 동안 A[R]의 키가 pivot 값 p 이상이면 R을 왼쪽으로 한 칸씩 이동시킨다. A[R]의 키가 p 미만이거나 R이 lo에 도달하면 이동을 멈춘다.
- 경계 조건: R이 lo에 도달한 경우에는 pivot 자신을 다시 검사하더라도 R을 더 감소시키지 않는다. 따라서 항상 R≥lo가 유지된다. 현재 정렬 구간에 p보다 작은 원소가 없다면 R은 lo에서 종료한다.

- 포인터 교환: L과 R의 위치를 비교한다. L<R이면 A[L]과 A[R]을 교환하고, L을 한 칸 증가시키고 R을 한 칸 감소시킨 다음 포인터 탐색을 반복한다.
- 탐색 종료 및 pivot 배치: L≥R이면 탐색을 종료하고 A[lo]와 A[R]을 교환하여 pivot을 인덱스 R에 배치한다. R=lo라면 pivot을 자기 자신과 교환하므로 위치는 변하지 않는다.
- Partition 결과: pivot 왼쪽에는 pivot보다 작은 키만 존재하고, pivot 오른쪽에는 pivot 이상의 키만 존재한다. 이후 pivot을 제외한 왼쪽 부분배열과 오른쪽 부분배열을 각각 재귀적으로 정렬한다. 길이가 0 또는 1인 부분배열은 추가 정렬 없이 종료한다.
- 중복 키가 많으면 분할이 심하게 불균형해질 수 있으며, 이 경우 세 구간(key < pivot, key = pivot, key > pivot)으로 나누는 3-way partition을 고려할 수 있다.
- **추가 메모리 거의 없음** → cache friendly

Pivot 선택 전략

전략	특징
First/Last	간단. **정렬된 입력에** $O(n^2)$
Middle	일부 경우에 잘 동작
Random	평균 좋으나 난수 생성 비용
Median-of-three	Median-of-three는 정렬 구간의 첫 원소, 중간 원소, 마지막 원소 중 키의 중앙값을 pivot으로 선택하는 전략이다. 이 방법은 일부 정렬되거나 편향된 입력에서 First/Last pivot보다 균형 잡힌 분할을 얻을 가능성을 높이지만, 모든 입력에 대해 균형 분할을 보장하지 않으며 최악 시간 복잡도 $O(n^2)$도 제거하지 못한다. 또한 pivot 선택 전략일 뿐이므로 동일 키의 상대 순서를 보존하는 안정성(stability)을 보장하지 않는다.

복잡도 유도

점화식

$$t(n) = t(|L|) + t(|R|) + d \cdot n$$

Best / Average ($|L| \approx |R| \approx n/2$):

$$t(n) = 2t(n/2) + dn = O(n \log n)$$

Worst ($|L| = 0$ 또는 $|R| = 0$, 예: 이미 정렬된 입력 + leftmost pivot):

$$t(n) = t(n-1) + dn = O(n^2)$$

복잡도 유도 — Quick Sort: Partition 단계에서 현재 부분배열의 모든 원소를 선형 시간에 처리하므로, 분할 비용은 Θ(*n*)이다. 길이가 0 또는 1인 부분배열에서는 재귀를 종료한다.

Best case — 균형 분할: 매 단계에서 두 부분배열의 크기가 거의 같으면

$T(n)=2T(n/2)+\Theta(n)$

따라서 $T(n)=\Theta(n \log n)$.

Average case — 무작위 pivot, 서로 다른 키: Pivot을 균등하게 무작위 선택하면, pivot보다 작은 원소가 *k*개일 확률은 각 $k=0,\cdots,n-1$에 대해 $1/n$이다.

$E[T(n)] = (1/n)\ \Sigma_{k=0}^{n-1}\ (E[T(k)] + E[T(n-1-k)]) + \Theta(n)$

이 점화식을 풀면 $E[T(n)]=\Theta(n \log n)$.

Worst case — 극단적으로 불균형한 분할: Pivot이 매번 최솟값이나 최댓값이면 크기가 0과 $n-1$인 두 부분배열로 분할된다.

$T(n)=T(n-1)+\Theta(n)$

따라서 $T(n)=\Theta(n^2)$. 예를 들어 서로 다른 키가 오름차순으로 정렬되어 있고 항상 첫 원소를 pivot으로 선택하면 이러한 최악의 분할이 반복된다.

8. Quick vs Merge vs Heap(힙)

비교

	Quick	Merge	Heap
Best	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Average	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Worst	$O(n^2)$	$O(n \log n)$	$O(n \log n)$
Extra space	$O(\log n)$ (stack)	$O(n)$	$O(1)$
In-place	✓	✗	✓
Stable	✗	✓	✗

실전 선택 기준

- **Quick sort**: 평균 시간 복잡도가 $O(n \log n)$이고 제자리 partition에서 좋은 메모리 접근 지역성을 보이는 경우가 많아 일반적인 배열 정렬에서 효율적인 선택이 될 수 있다. 그러나 실제 성능은 pivot 선택 전략, 입력 분포, 구현 최적화와 하드웨어 특성에 따라 달라지므로 항상 가장 빠른 정렬 알고리즘이라고 단정할 수는 없다. 안정성이 필요한 경우에는 stable Merge sort를 고려하고, 최악 시간 $O(n \log n)$ 보장과 $O(1)$ 추가 배열 공간이 중요한 경우에는 Heap sort를 고려할 수 있다. 또한 실제 라이브러리에서는 여러 알고리즘을 결합한 하이브리드 정렬을 사용할 수 있으므로 요구사항과 구현 특성에 따라 선택해야 한다.
- **Merge sort**: **worst-case 보장** + stable 필요 시 (예: 외부 정렬, linked list)
- **Heap sort**: **실시간 시스템** (worst $O(n \log n)$ 보장 + $O(1)$ 공간)
  - quick sort는 worst $O(n^2)$이라 실시간엔 위험

Quick space 주의: Quick sort의 일반적인 재귀 구현은 균형 잡힌 분할이 이루어질 때 호출 스택 깊이가 $O(\log n)$이지만, 극단적으로 불균형한 분할이 반복되면 스택 깊이가 $O(n)$까지 증가할 수 있다. 최악의 경우에도 추가 스택 공간을 $O(\log n)$으로 제한하려면 매 단계에서 크기가 더 작은 부분배열만 재귀 호출하고 더 큰 부분배열은 현재 함수의 반복문으로 처리하는 tail-recursion elimination을 적용해야 한다. 단순히 작은 쪽을 먼저 재귀 호출하는 것만으로는 이러한 최악 공간 상한이 보장되지 않는다. 따라서 기본 Quick sort의 추가 공간은 평균 $O(\log n)$, 최악 $O(n)$이며, 위 최적화를 적용한 구현에서는 최악 스택 공간도 $O(\log n)$이다.

Quick의 cache 이점

- Quick sort는 제자리 partition 과정에서 같은 배열의 비교적 인접한 원소들을 반복적으로 접근하기 때문에 좋은 공간 지역성(cache locality)을 보이는 경우가 많으며, 이러한 메모리 접근 특성이 실제 성능에 유리하게 작용할 수 있다.
- 반면 배열 기반 Merge sort는 보조 버퍼가 필요하지만 병합 과정에서 원소들을 순차적으로 읽고 쓰므로 캐시 효율이 반드시 나쁜 것은 아니다. 두 알고리즘의 실제 cache miss 횟수와 실행 성능은 배열 크기,

메모리 계층 구조 및 구현 방식에 따라 달라진다. 여기서 사용하는 개념은 cache coherence가 아니라 cache locality이다.

Summary

Algorithm	Best	Avg	Worst	Space	Stable	Method
Bubble	n	n^2	n^2	1	✓	Adjacent swap
Selection	n^2	n^2	n^2	1	✗	Min-search
Insertion	n	n^2	n^2	1	✓	Insert into sorted
Merge	$n \log n$	$n \log n$	$n \log n$	n	✓	D&C (merge)
Quick	$n \log n$	$n \log n$	n^2	$\log n$	✗	D&C (partition)
Heap	$n \log n$	$n \log n$	$n \log n$	1	✗	Heap
Bucket (정수 키 직접 인덱싱)	$n+N$	$n+N$	$n+N$	$n+N$	✓ (버킷 내부 순서 보존 시)	Non-comparison
Bucket (실수 키 구간 분할)	$n+k$	$n+k+n^2/k$ (독립 균등분포 가정)	n^2+k	$n+k$	✓ (입력 순서대로 bucket에 추가하고 bucket 내부를 안정 정렬할 때)	Non-comparison

실수 키 구간 분할 Bucket Sort의 시간 복잡도는 k개의 bucket을 초기화하고 각 bucket 내부를 Insertion Sort로 정렬하는 구현을 기준으로 한다. 최선 시간은 $O(n+k)$이며 기대값으로 표현하지 않는다. n개 키가 독립적인 균등분포를 따를 때 평균 시간 상한은 $O(n+k+n^2/k)$이고, $k=\Theta(n)$이면 $O(n)$으로 단순화된다. 입력이 특정 bucket에 집중되면 최악 시간은 $O(n^2+k)$가 된다.

실수 키 구간 분할 Bucket Sort에서는 같은 키의 원소들이 반드시 같은 bucket에 배치된다. Scatter 단계에서 각 bucket에 입력 순서대로 원소를 추가하고, bucket 내부를 안정적인 정렬 알고리즘으로 정렬하며, gather 단계에서 정렬된 bucket 내부 순서를 유지하면 전체 정렬도 안정적이다. 예를 들어 원소를 순서대로 bucket의 끝에 추가한 뒤, 같은 키의 기존 원소를 뒤로 밀지 않는 Insertion Sort로

bucket 내부를 정렬하면 안정성을 유지할 수 있다. 이러한 조건을 만족하지 않는 구현은 안정성을 보장하지 않는다.

* float key 변형처럼 bucket 내부 정렬(insertion sort)이 필요한 경우, 한 bucket에 몰리면 내부 정렬 비용 **$\Theta(n^2)$**이 지배. 정수 key 직접 index 버전은 worst에도 O(n+N).

Summary 표 설명 및 Heap Sort 최선 시간 주의: 이 표의 Best, Avg, Worst는 각각 최선·평균·최악 입력에서의 점근적 시간 상한 $O(\cdot)$을 나타내며, Space는 입력 배열을 제외한 추가 공간의 점근적 상한을 나타낸다. n은 정렬할 원소의 개수이고, N은 정수 키를 직접 인덱싱하는 방식에서 사용되는 키 범위의 크기이며, k는 실수 키 구간 분할 방식에서 사용하는 bucket의 개수이다. 표의 1, n, $n \log n$, n^2 표기는 각각 $O(1)$, $O(n)$, $O(n \log n)$, $O(n^2)$을 의미하며, 별도로 Θ 기호를 명시하지 않은 경우에는 반드시 타이트한 상·하한을 뜻하지 않는다.

Heap Sort의 Best 표기: Heap Sort의 Best 셀에 기재된 $n \log n$은 $O(n \log n)$이라는 점근적 상한을 의미하므로, 모든 입력 조건에서 최선 시간이 반드시 $\Theta(n \log n)$이라는 단정은 아니다. Bottom-up heap construction과 동률에서 불필요한 sift-down을 중단하는 일반적인 Max-Heapify 구현을 기준으로, 모든 키가 동일한 입력에서는 힙 구성에 $\Theta(n)$ 시간이 걸리고 각 원소 추출 후의 sift-down이 상수 시간에 종료되므로 전체 시간이 $\Theta(n)$이 될 수 있다. 반면 모든 키가 서로 다른 경우에는 일반적인 Heap Sort의 최선 시간도 $\Theta(n \log n)$이다. 따라서 시험에서 최선 시간의 타이트한 복잡도를 요구한다면 키의 중복 허용 여부와 동률 처리 구현을 구별해야 한다. Heap Sort의 최악 시간은 $\Theta(n \log n)$이며, 반복형 sift-down을 사용하는 제자리 구현의 추가 공간은 $O(1)$이다.

기타 공간 복잡도 주의: Quick Sort의 추가 공간은 기본 재귀 구현에서 평균 $O(\log n)$, 최악 $O(n)$이며, 작은 부분배열만 재귀 호출하고 큰 부분배열을 반복문으로 처리하도록 최적화하면 최악에도 $O(\log n)$으로 제한할 수 있다. Bucket Sort는 정수 키 직접 인덱싱 방식과 실수 구간 분할·내부 정렬 방식을 구별하여 별도 행으로 표시한다.

- **Comparison sort의 하한**: 모든 비교 기반 정렬 알고리즘에는 최악 $\Omega(n \log n)$의 하한이 존재한다. 입력 순열이 균등하게 주어진 경우 평균 비교 횟수에도 $\Omega(n \log n)$ 하한이 성립하지만, 개별 알고리즘의 최선 시간까지 반드시 $\Omega(n \log n)$이어야 하는 것은 아니다.
- Bucket sort와 Radix sort는 키끼리의 대소 비교만으로 정렬 순서를 결정하지 않기 때문에 비교 정렬의 $\Omega(n \log n)$ 하한이 직접 적용되지 않는다. 다만 선형 시간에 정렬할 수 있는 조건은 알고리즘마다 다르다. 정수 키 직접 인덱싱 Bucket sort는 키 범위 크기가 N일 때 $O(n+N)$ 시간이 걸리고, 실수 구간 분할 Bucket sort는 적절한 bucket 수와 독립적인 균등분포 가정 아래 기대 $O(n)$ 시간이 가능하다. Radix sort는 키를 d개의 자릿수로 표현하고 각 자리마다 기수 b의 안정적인 Counting sort를 수행하면 $O(d(n+b))$ 시간이 걸리며, $d=O(1)$이고 $b=O(n)$인 조건에서는 $O(n)$ 시간이 가능하다. Radix sort의 이 시간 상한은 키 분포가 균등하다는 가정을 필요로 하지 않는다.